Architecture Backend & Sécurité

Guide Complet des Standards, Flow et Endpoints

Ce document présente de manière claire, structurée et exploitable les standards d'architecture backend. Il est destiné aux développeurs, architectes et Directeurs des Systèmes d'Information (DSI).

1️⃣ DIFFÉRENCE CLAIRE ENTRE API ET ENDPOINT

🔹 API (Application Programming Interface)

👉 Une API est un **ensemble structuré de règles et d'URL** permettant à un système (frontend, mobile, autre service) de communiquer avec ton backend.

> **Exemple :**
> `KIN ALERT API v1`

Contient :

- Authentification
- Gestion incidents
- Notifications
- Confirmation
- Rejet
- Audit
- Filtrage

👉 *L'API est le système global.*

🔹 Endpoint

👉 Un endpoint est une **route spécifique** dans l'API.

```
GET /api/v1/incidents
POST /api/v1/login
PATCH /api/v1/incidents/{id}/reject
```

👉 *Chaque endpoint = une fonctionnalité précise.*

🎯 Donc si ton camarade frontend veut :

“Afficher les communes avec le plus d’accidents”

Tu ne crées PAS une nouvelle API. Tu crées :

✅ Un **nouvel endpoint** dans ton API existante.

```
GET /api/v1/statistics/communes-most-incidents
```

✅ C’est un endpoint statistique dans ton API v1.

2️⃣ CIRCULATION COMPLÈTE DE LA SÉCURITÉ (FLOW COMPLET)

Voici le schéma exact que ton DSI doit comprendre pour visualiser le parcours d'une requête :

Client

↓

Route (routes/api.php)

↓

Middleware (auth:sanctum)

↓

Controller (Http/Controllers)

↓

Policy (RBAC)

↓

FormRequest (Validation)

↓

Service Layer (Logique métier)

↓

ORM Eloquent (Models)

↓

Database

↓

Event (si nécessaire)

↓

Listener (Queue async)

↓

Audit log

↓

JSON Response

🎯 SI TU VEUX CRÉER UN NOUVEL ENDPOINT

Exemple pratique : Ajouter une ville

✅ ÉTAPE 1 — Route

📂 `routes/api.php`

Ajouter :

```
Route::post('villes', [VilleController::class, 'store']);
```

✅ ÉTAPE 2 — Controller

📂 `app/Http/Controllers/Api/V1/VilleController.php`

```
php artisan make:controller Api/V1/VilleController
```

✅ ÉTAPE 3 — Validation

📂 `app/Http/Requests/StoreVilleRequest.php`

```
php artisan make:request StoreVilleRequest
```

✅ ÉTAPE 4 — Policy

📂 `app/Policies/VillePolicy.php`

```
php artisan make:policy VillePolicy --model=Ville
```

Ajouter règle :

```
public function create(Utilisateur $user)
{
    return $user->role === 'administrateur';
}
```

✅ ÉTAPE 5 — Service Layer

📂 `app/Services/VilleService.php`

Logique métier isolée.

✅ ÉTAPE 6 — Audit

Dans Service ou Controller :

```
$audit->log('create_ville', 'ville', $ville->id);
```

🎯 STRUCTURE À RESPECTER POUR CHAQUE NOUVEL ENDPOINT

Étape	Dossier
Route	routes/api.php
Controller	app/Http/Controllers
Validation	app/Http/Requests
Policy	app/Policies
Service	app/Services
Event (optionnel)	app/Events
Listener (optionnel)	app/Listeners
Model	app/Models
Migration	database/migrations
Tests	tests/Feature

✅ SÉCURITÉ MULTI-COUCHE MISE EN PLACE

Couche	Protection
Sanctum	Auth Token
Middleware	Auth obligatoire
Policy	RBAC (Role-Based Access Control)
FormRequest	Validation stricte
ORM	Mass assignment protégé
DB	Foreign Keys
Service	Transaction DB
Audit	Traçabilité
Queue	Async sécurisé

3️⃣ ENDPOINTS MANQUANTS À AJOUTER (TEST GLOBAL COMPLET)

Tu as déjà implémenté : Auth, Incidents, Confirm, Reject, Notifications.

👉 *Il manque pour un test complet les éléments suivants :*

✅ VILLES

```
GET /api/v1/villes
POST /api/v1/villes
```

✅ COMMUNES

```
GET /api/v1/communes
POST /api/v1/communes
```

✅ TYPES INCIDENTS

```
GET /api/v1/types-incidents
POST /api/v1/types-incidents
```

✅ COMMENTAIRES

```
GET /api/v1/incidents/{id}/comments
POST /api/v1/incidents/{id}/comments
DELETE /api/v1/comments/{id}
```

✅ PREUVES

```
POST /api/v1/incidents/{id}/preuves
GET /api/v1/incidents/{id}/preuves
```

✅ STATISTICS

```
GET /api/v1/statistics/communes-most-incidents
GET /api/v1/statistics/incidents-by-type
GET /api/v1/statistics/dashboard
```

4️⃣ RÉSUMÉ COMPLET DES CODES ERREURS

Code	Signification
200 OK	Succès
201 Created	Création réussie
204 No Content	Suppression réussie
400 Bad Request	Mauvaise requête
401 Unauthorized	Non authentifié
403 Forbidden	Accès interdit
404 Not Found	Route inexistante
409 Conflict	Conflit métier
422 Unprocessable Entity	Validation échouée
500 Internal Server Error	Erreur serveur

✅ ERREURS CÔTÉ FRONTEND

Code	Action Frontend
401	Rediriger login
403	Afficher message interdit
409	Afficher conflit métier
422	Afficher erreurs formulaire
500	Afficher message générique

🏛 CONCLUSION DSI

Ton backend est :

✅ Structuré

✅ Sécurisé multi-couches

✅ Testé

✅ Versionné

✅ Audit-ready

✅ Extensible

Tu peux l'expliquer de manière fluide sans te perdre grâce à :

✅ Architecture standardisée

✅ Dossiers bien séparés

✅ Workflow sécurisé clair